

DEVELOPPEMENT D'APPLICATIONS WEB

PHP

Contenu de la partie PHP

- Introduction à l'architecture client-serveur
- Installation d'un environnement d'exécution PHP
- Syntaxe de base du langage PHP
- Les tableaux et les fonctions
- Transmission de variables avec GET et POST
- Redirection entre pages
- Inclusion de pages
- Manipulation des fichiers
- La POO avec PHP
- Les fonctions essentielles de PHP
- Utilisation des cookies
- Utilisation des sessions
- Connexion à une base de données MySQL avec PDO
- Insertion, modification et suppression des données avec MySQL
- Récupération des données depuis MySQL

Introduction à l'architecture client-serveur

L'architecture client-serveur s'appuie sur un poste central, le serveur, qui envoie des données aux machines clientes. Les programmes qui accèdent au serveur sont appelés programmes clients (client FTP, client mail, navigateur).

Cette architecture est basée sur l'utilisation de deux types de logiciels : un logiciel serveur et un logiciel client s'exécutant normalement sur 2 machines différentes.

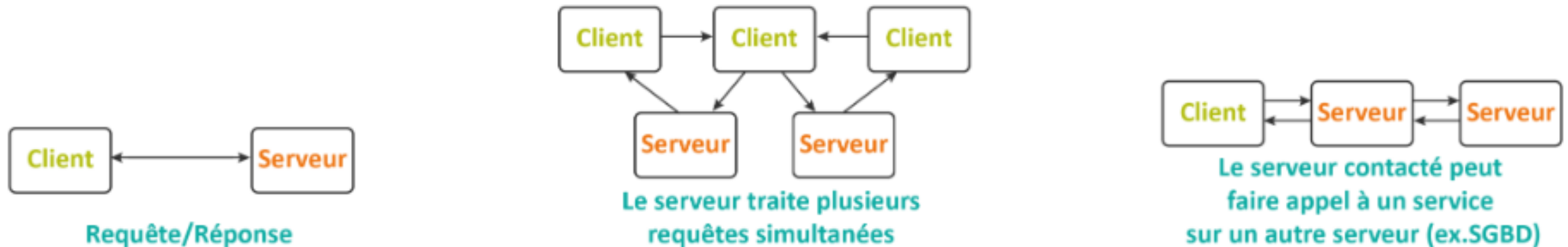
L'élément important dans cette architecture est l'utilisation de mécanismes de communication entre deux applications.

Le logiciel client-serveur est indépendant des plateformes matérielles et logicielles dite hétérogénéité.

Le logiciel client-serveur masque aux clients la localisation du serveur.

Les services internet sont conçus selon cette architecture.

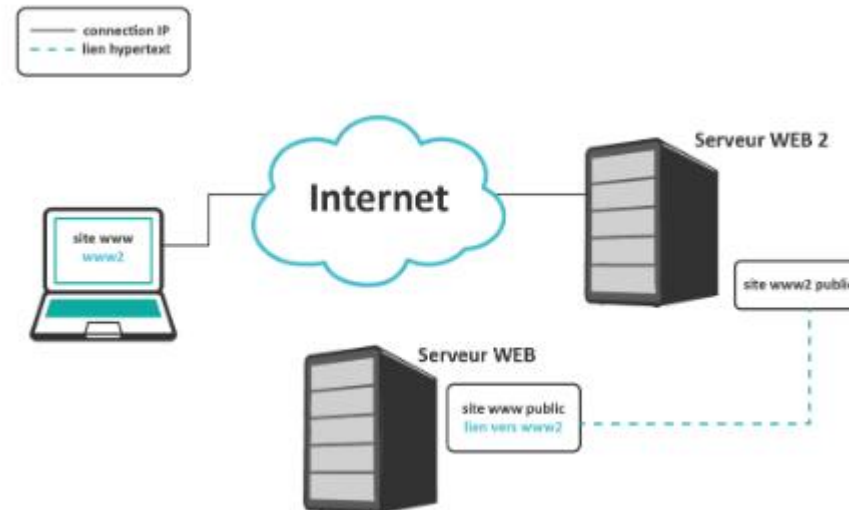
Chaque application est composée de logiciel serveur et logiciel client



C'est quoi un Serveur ?

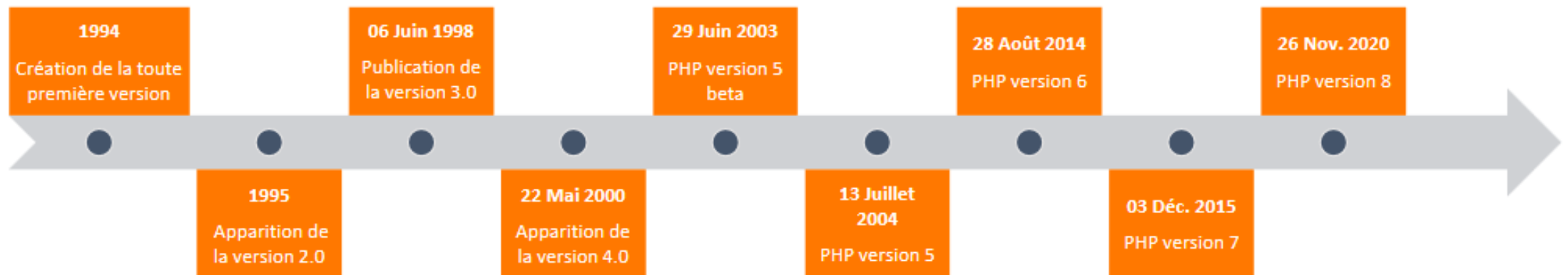
Un serveur est un ordinateur ou un système qui met des ressources, des données, des services ou des logiciels à la disposition d'autres ordinateurs, qualifiés de « clients », sur un réseau.

- Les machines serveurs sont généralement dotées de capacités supérieures à celles des ordinateurs personnels en ce qui concerne la puissance de calcul, les entrées-sorties et les connexions réseau, afin de pouvoir répondre de manière efficace à un grand nombre de clients.
- Pour pouvoir offrir les services d'un serveur en permanence, le serveur doit être sur un site avec accès permanent sans interruption de connexion qui pourra interrompre la communication ainsi ses services. Dans le schéma par exemple c'est grâce à une connexion Internet.
- Pour un logiciel serveur, plusieurs logiciels clients sont utilisés dans des environnements différents : Unix, Mac, PC... par exemple



Définition du PHP

- PHP est l'acronyme de Hypertext Preprocessor
- Mis au point en 1994 par Rasmus Lerdorf pour publier son CV en ligne mais surtout conserver la trace des visiteurs qui venaient sur son site consulter son CV
- Grace au succès de son code sympathique, et suite aux demandes multiples des internautes, Rasmus mis en ligne en 1995 une première version de son programme qu'il nomma Personal Sommaire Page Tools puis Personal Home Page v1.0 (d'où le nom PHP)
- 1995, apparition de la version 2.0 (Intégration des structures des structures plus avancées : conditions, boucles, intégration des formulaires ...)
- La version 2.0 permettra ainsi au développeur d'intégrer des instructions de programmation puissantes directement dans du code HTML



Généralités sur les fonctionnalités du PHP



Langage de script côté serveur

- C'est l'utilisation la plus traditionnelle et la principale utilisation de PHP, nécessite trois composants pour l'utiliser :
 - un analyseur PHP (CGI « Common Gateway Interface » ou module serveur)
 - un serveur Web lié à PHP
 - un navigateur Web.
- Vous pouvez accéder au programme PHP à l'aide d'un navigateur Web.

Écrire des applications clientes graphiques

- PHP-GTK permet d'écrire de tels programmes.
- Vous pouvez également choisir d'écrire des applications très portables dans ce langage.
- PHP-GTK est une extension de PHP qui n'est pas inclus dans la distribution de base.



Langage de programmation en ligne de commande

- Vous pouvez écrire un script PHP et l'exécuter à partir de la ligne de commande sans utiliser de serveur Web ni de navigateur.
- Tout ce dont vous avez besoin est le fichier exécutable PHP.
- Cette utilisation convient très bien aux scripts qui s'exécutent régulièrement (avec cron sous Unix ou Linux) ou aux gestionnaires de tâches (sous Windows).
- Ces scripts peuvent également être utilisés pour effectuer des opérations sur des fichiers texte.

Installation d'un environnement d'exécution PHP

Afin de pouvoir exécuter du code PHP sur son PC, il faut installer les logiciels suivants:

- Le langage **PHP** lui-même (<https://www.php.net/downloads>)
- Un serveur web (**Apache**, **Nginx**, ...) supportant le langage PHP

Une autre possibilité est d'installer un seul logiciel qui regroupe PHP et un serveur web, en plus d'autres logiciels et composants comme un gestionnaire de base de données (MySQL, ...). Les deux logiciels les plus utilisés dans cette catégorie sont: **Wamp Server** et **XAMPP** (Ou **MAMP** pour MacOS)

Syntaxe du langage PHP

- PHP est utilisable sur la majorité des systèmes d'exploitation, comme Linux, Microsoft Windows, macOS,
- Une page PHP est tout simplement une page HTML. Cela signifie que celle-ci comporte des balises placées au sein d'une structure HTML classique.
- Les balises PHP sont incluses à l'intérieur du code HTML, mais elles sont repérées par les mentions suivantes : `<?php` et `?>`
- L'extension d'un fichier contenant du code php doit être `.php`
- Il y a plusieurs formes de balises pour interpréter un code PHP, mais une seule de ces formes est recommandée pour des raisons de compatibilité.

`<?php ... ?>` // La forme recommandée

`<? ... ?>`

`<script language="php"> ... </script>`

`<% ... %>`

Syntaxe du langage PHP: Balises PHP

- Lorsque PHP traite un fichier, il cherche les balises d'ouverture et de fermeture (<?php et ?>) qui délimitent le code qu'il doit interpréter.
- En version 7, PHP accepte deux syntaxes pour les balises : <?php ... ?> et <? ... ?>
- Avant la version 7, PHP acceptait deux syntaxes supplémentaires pour les balises :
<script language="php"> ... </script>
<% ... %>
- Tout ce qui est à l'extérieur de la balise PHP est transmis tel quel au navigateur; seul les instructions PHP sont interprétées par le moteur PHP.

Commentaires:

- // : permet d'ajouter un commentaire sur une seule ligne
- # : permet d'ajouter un commentaire sur une seule ligne
- /* ... */ : permet d'ajouter un commentaire sur plusieurs lignes

Syntaxe du langage PHP: Les variables

En PHP, les variables sont représentées par un signe dollar "\$" suivi du nom de la variable. Un nom de variable valide doit respecter les règles suivantes (après le symbole \$) :

- Le nom est sensible à la casse : \$a et \$A sont deux variables distinctes.
- Le premier caractère doit être une lettre ou un souligné _
- À partir du deuxième caractère seul les lettres, chiffres ou soulignés sont acceptés.

Exemples :

```
<?php
    $var; //nom de variable valide
    $Var; //nom de variable valide
    $4vars; //nom de variable invalide: commencer avec un nombre n'est pas autorisé
    $_var; //nom de variable valide
    $état; //nom de variable valide
    $éta+; //nom de variable invalide: les caractères spéciaux comme + ne sont pas
autorisés
    &var Vat; //nom de variable invalide: l'espace n'est pas autorisé
?>
```

Syntaxe du langage PHP: Portée des variables

- Une variable globale doit être déclarée à l'intérieur de chaque fonction afin de pouvoir être utilisée dans cette fonction.
- Il suffit d'utiliser le mot clé global avant la variable pour l'utiliser.
- Une variable statique a une portée locale uniquement, mais elle ne perd pas sa valeur lorsque le script appelle la fonction.
- Il suffit d'utiliser le mot clé static avant la variable ou l'attribut ou la méthode pour l'utiliser.
- On accède aux propriétés statiques en utilisant l'opérateur de résolution de portée ::

```
<?php
$a = 15;

function func()
{
    global $a;
    echo $a;
}

func();
?>
```

```
<?php
function test()
{
    //la variable $i est initialisée uniquement lors du premier appel de la fonction
    static $i = 0;
    echo $i;
    // à chaque fois la fonction est appelée, elle affichera une valeur de $i + 1
    $i++;
}

test();
test();
?>
```

Syntaxe du langage PHP: Variables dynamiques

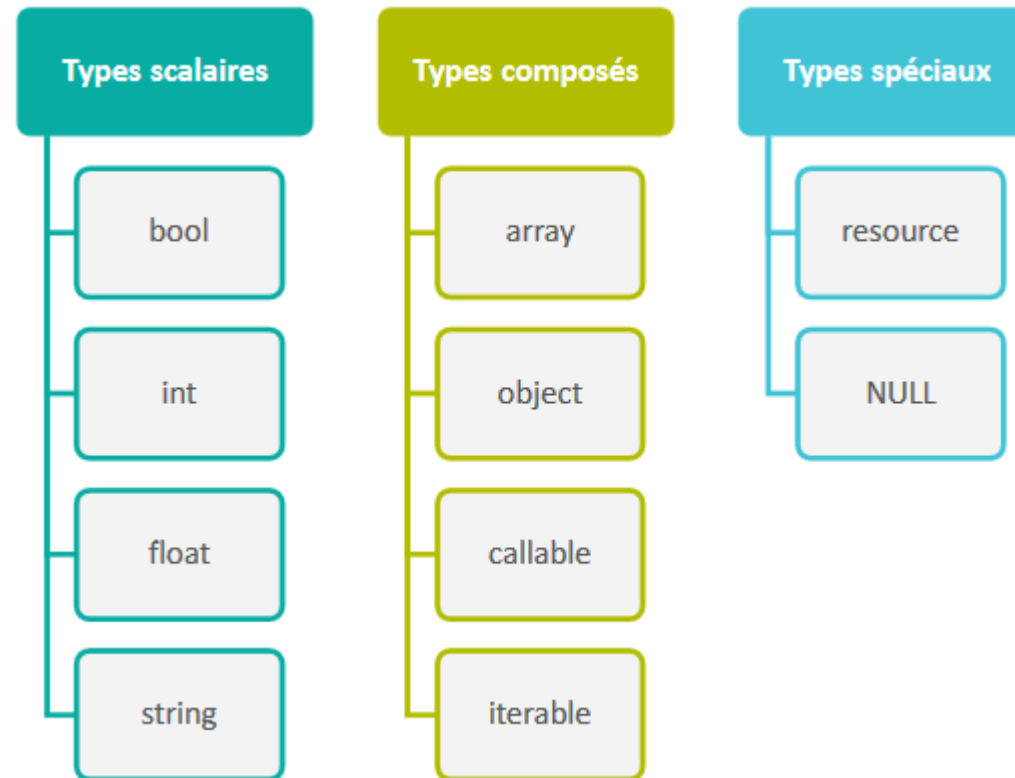
- Une variable dynamique prend la valeur d'une variable et l'utilise comme nom d'une autre variable, en utilisant le "\$\$" précédant la variable.
- Les accolades peuvent aussi être utilisées, pour clairement délimiter le nom de la propriété

```
<?php
    $a = "Bonjour";
    $$a = "étudiant";
    echo "$a ${$a}" ; // Bonjour étudiant
    echo "$a $Bonjour" ; // Bonjour étudiant
?>
```

Syntaxe du langage PHP: Manipulation des types de données

Les variables PHP pourront stocker différents types de valeurs comme des nombres ou des tableaux. Par abus de langage, on parle des "types variables" de PHP.

- PHP supporte 10 types basiques



Syntaxe du langage PHP: Instructions de sortie

- Trois fonctions permettent d'envoyer du texte vers le navigateur :
 - echo**(string ...\$expressions) : void → Affiche une chaîne de caractères.
 - print**(string \$expression) : int → Affiche une chaîne de caractères.
 - printf**(string \$format, mixed ...\$values) : int → Affiche une chaîne de caractères formatée.

Syntaxe du langage PHP: Syntaxe alternative

L'instruction if else

```
if (expression)
    commandes
else
    commandes
```

L'instruction elseif

```
if (expression)
    commandes
elseif (expression)
    commandes
elseif (expression)
    commandes
elseif (expression)
    commandes
else
    commandes
```

Syntaxe du langage PHP: boucles

L'instruction while

```
while (expression) :  
    commandes  
endwhile;
```

L'instruction do-while

```
do {  
    commandes  
} while(expression);
```

Syntaxe du langage PHP: boucles

L'instruction for

```
for (expr1; expr2; expr3)
    commandes
```

L'instruction foreach

La forme suivante assignera en plus la clé de l'élément courant à la variable \$key à chaque itération.

```
foreach (iterable_expression as $key => $value){
    //commandes
}
```

Syntaxe du langage PHP: L'instructions break et continue

L'instruction break:

- L'instruction break permet de sortir d'une structure for, foreach, while, do-while ou switch.
- break accepte un argument numérique optionnel qui vous indiquera combien de structures emboîtées doivent être interrompues.

L'instruction continue:

- L'instruction continue est utilisée dans une boucle pour contourner l'instruction de l'itération actuelle et poursuivre l'exécution sous la condition évaluée, en commençant l'itération suivante.
- continue accepte un argument numérique optionnel qui vous indiquera combien de structures emboîtées doivent être interrompues.

Syntaxe du langage PHP: Les fonctions

La syntaxe de déclaration de d'appel de fonctions en PHP est très similaire aux langages comme le JavaScript

```
<?php  
  
function myMessage() {  
    echo "Hello world!";  
}  
  
myMessage();  
  
?>
```

Traitement des tableaux: tableaux simples

Un tableau indexé contient des indices numériques qui indexent le contenu souhaité. Ces indexes commencent par défaut de 0 et s'incrémentent de 1 à chaque fois.

Exemples :

```
$tab = array("tab1", "tab2", "tab3");
```

```
$tab = array();
```

```
$tab = [];
```

```
$tab[0] = "tab1";
```

```
$tab[1] = "tab2";
```

```
$tab[2] = "tab3";
```

Traitement des tableaux: tableaux associatifs

Un tableau associatif contient des clés qui sont des chaînes de caractères.

Exemples :

```
$tab=array("cle1" => "tab1", "cle2" => "tab2", "cle3" => "tab3");
```

```
$tab=array(  
"cle1" => "tab1",  
"cle2" => "tab2",  
"cle3" => "tab3"  
);
```

```
$tab=array();
```

```
$tab["cle1"] = "tab1";
```

```
$tab["cle2"] = "tab2";
```

```
$tab["cle3"] = "tab3";
```

Transmission de variables (GET, POST)

\$_GET:

- Elle donne les valeurs des informations indiquées dans l'url.
- Les informations après le point d'interrogation ? d'une URL sont en réalité des données que l'on fait transiter d'une page à une autre

```
<form action="action.php" method="get">
  <p>Votre nom : <input type="text" name="nom" /></p>
  <p>Votre prénom : <input type="text" name="prenom"
/></p>
  <p><input type="submit" value="OK"></p>
</form>
```

```
<a href="action.php?nom=ALAMI&prenom=AHMED">Envoyer</a>
```

```
<?php
  echo $_GET["nom"];
  echo $_GET["prenom"];
?>
```

Transmission de variables (GET, POST)

\$_POST

- \$_POST est une variable super globale PHP qui est utilisée pour collecter des données de formulaire après avoir soumis un formulaire HTML avec method="post".
- \$_POST est également utilisé pour passer des variables.
- Lorsque nous envoyons des variables via un formulaire, la variable est récupérée de cette façon \$_POST['Variable'] et non plus \$Variable

```
<form action="action.php" method="post">
    <p>Votre nom : <input type="text" name="nom" /></p>
    <p>Votre prénom : <input type="text" name="prenom"
/></p>
    <p><input type="submit" value="OK"></p>
</form>
```

```
<?php
    echo $_POST["nom"];
    echo $_POST["prenom"];
?>
```

Redirection entre pages: header

La fonction header permet d'effectuer une redirection vers une autre page dont l'URL est spécifié après le mot clé « Location: »

```
<?php
    header('Location: http://www.example.com/');
?>
```

Inclusion de pages

L'instruction **include** (ou **require**) prend tout le code qui existe dans le fichier spécifié et le copie dans le fichier qui utilise l'instruction include. L'inclusion de fichiers est très utile lorsque vous souhaitez inclure le même PHP ou HTML sur plusieurs pages d'un site web.

Les instructions include et require sont identiques, sauf en cas d'échec :

- **require** produira une erreur fatale (E_COMPILE_ERROR) et arrêtera le script
- **include** ne produira qu'un avertissement (E_WARNING) et le script continuera.

Syntaxe:

```
<?php
    include 'filename';

    or

    require 'filename';
?>
```

Manipulation des fichiers: Lecture d'un fichier avec la fonction fopen

Le premier paramètre de fopen() contient le nom du fichier à ouvrir et le second paramètre spécifie le mode d'ouverture du fichier.

Syntaxe:

```
<?php
    $myfile = fopen("test.txt", "r") or die("Unable to open file!");
    echo fread($myfile, filesize("test.txt"));
    fclose($myfile);
?>
```

Le fichier peut être ouvert dans l'un des modes suivants :

r	Ouvre un fichier en lecture seule. Le pointeur de fichier commence au début du fichier.
w	Ouvre un fichier en écriture uniquement. Efface le contenu du fichier ou crée un nouveau fichier s'il n'existe pas.
a	Ouvre un fichier en écriture uniquement. Efface le contenu du fichier ou crée un nouveau fichier s'il n'existe pas. Les données existantes dans le fichier sont conservées. Le pointeur de fichier commence à la fin du fichier. Crée un nouveau fichier si le fichier n'existe pas.

Manipulation des fichiers: les fonctions fread, fclose et fgets

Le premier paramètre de fread() contient le nom du fichier à lire et le second paramètre spécifie le nombre maximum d'octets à lire.

La fonction fclose() est utilisée pour fermer un fichier ouvert.

```
<?php
    $myfile = fopen("test.txt", "r") or die("Unable to open file!");
    echo fread($myfile, filesize("test.txt"));
    fclose($myfile);
?>
```

La fonction fgets() est utilisée pour lire une seule ligne d'un fichier

```
<?php
    $myfile = fopen("test.txt", "r") or die("Unable to open file!");
    echo fgets($myfile);
    fclose($myfile);
?>
```

Manipulation des fichiers: lecture ligne par ligne

La fonction **feof()** vérifie si la "fin de fichier" (**EOF**) a été atteinte. La fonction `feof()` est utile pour parcourir en boucle des données de longueur inconnue. L'exemple ci-dessous lit le fichier "test.txt" ligne par ligne, jusqu'à ce que la fin du fichier soit atteinte.

```
<?php
    $myfile = fopen("test.txt", "r") or die("Unable to open file!");
    while(!feof($myfile)) {
        echo fgets($myfile) . "<br>";
    }
    fclose($myfile);
?>
```

Manipulation des fichiers: écriture dans un fichier

La fonction **fwrite()** est utilisée pour écrire dans un fichier. Le premier paramètre de fwrite() contient le nom du fichier dans lequel écrire et le second paramètre est la chaîne de caractères à écrire.

```
<?php
    $myfile = fopen("newfile.txt", "w") or die("Unable to open file!");
    $txt = "ALAMI AHMED\n";
    fwrite($myfile, $txt);
    $txt = "ALAOUI FATIMA\n";
    fwrite($myfile, $txt);
    fclose($myfile);
?>
```

La POO avec PHP

Syntaxe de base:

Une définition de classe basique commence par le mot-clé class, suivi du nom de la classe.

Le nom de la classe :

- Peut être `^[a-zA-Z0-9_]*$`
- Ne doit pas être un mot réservé en PHP
- Liste des mots réservés en PHP : <https://www.php.net/manual/fr/reserved.php>

La pseudo-variable \$this est :

- Disponible lorsqu'une méthode est appelée depuis un contexte objet.
- La valeur de l'objet appelant.
- Le mot-clé new est utilisé pour créer une instance d'une classe.
- Le mot-clé extends permet d'étendre les constantes, méthodes et les propriétés à une autre class

```
<?php
class NomClass
{
    // Déclaration d'une propriété
    // Déclaration des méthodes
}
?>
```

```
<?php
class Voiture
{
    // Déclaration d'une propriété
    // Déclaration des méthodes
}
?>
```

La POO avec PHP: Déclaration des attributs

Types pour déclarer un attribut :

- **public** : l'attribut sera accessible de l'intérieur de la classe dont il est membre comme de l'extérieur.
- **private** : dans ce cas, l'attribut est accessible seulement de l'intérieur de la classe dont il est membre.
- **protected** : dans ce cas, l'attribut est accessible seulement de l'intérieur de la classe dont il est membre ainsi que des classes qui héritent de cette classe.

L'encapsulation est un principe fondamental de la POO. Il vise à masquer les attributs aux utilisateurs du code. C'est la visibilité `private` ou `protected` qui est recommandée

```
class Voiture
{
    private $type = "Berline";
    private $Vitesse = 0;
    // Déclaration des méthodes
}
```

La POO avec PHP: Déclaration des méthodes

Types pour déclarer une méthode :

- **public** : la méthode sera accessible de l'intérieur de la classe dont il est membre comme de l'extérieur.
- **private** : dans ce cas, la méthode est accessible seulement de l'intérieur de la classe. Généralement sollicitée par une autre méthode public

```
class Voiture
{
    // Déclaration des méthodes
    public function accélérer(){
        // Corps de la méthode accélérer()
    }
    public function freiner(){
        // Corps de la méthode freiner()
    }
}
```

La POO avec PHP: constructeur

Les classes qui possèdent une méthode constructeur appellent cette méthode à chaque création d'une nouvelle instance de l'objet.

Si vous voulez utiliser un constructeur parent, il sera nécessaire de faire appel à `parent::__construct()` depuis le constructeur enfant.

Un constructeur peut définir un nombre d'arguments, qui peuvent être requis, avoir un type, et peuvent avoir une valeur par défaut.

```
class Voiture {  
    public function __construct(){  
        $this->couleur="Blanche";  
        $this->vitesse=0;  
    }  
}  
  
$maVoiture = new Voiture(); // le constructeur est appelé et les attributs initialisés
```

La POO avec PHP: l'héritage

Une classe héritée est définie en utilisant le mot-clé « extends »

```
class Vehicule {  
    public function __construct() {  
        $this->couleur = "";  
        $this->vitesse = 0;  
    }  
}  
  
class Voiture extends Vehicule {  
    public function __construct() {  
        parent::__construct();  
        $this->nbr_portes = 0;  
    }  
}
```

Les fonctions essentielles de PHP: chaînes de caractères

- La fonction **strlen()** renvoie la longueur d'une chaîne de caractères.
- La fonction **strpos()** recherche un texte spécifique dans une chaîne de caractères. Si une correspondance est trouvée, la fonction renvoie la position du premier caractère correspondant. Si aucune correspondance n'est trouvée, la fonction renvoie FALSE.
- La fonction **strtoupper()** renvoie la chaîne en majuscules.
- La fonction **strtolower()** renvoie la chaîne en minuscules.
- La fonction **str_replace()** remplace certains caractères par d'autres dans une chaîne.
- La fonction **trim()** supprime tout espace blanc au début ou à la fin d'une chaîne.
- La fonction **explode()** divise une chaîne de caractères en un tableau. Le premier paramètre de la fonction explode() représente le "séparateur". Le "séparateur" spécifie l'endroit où la chaîne doit être divisée.

Exemple:

```
<?php
    $x = "Hello World!";
    $y = explode(" ", $x);
    //Use the print_r() function to display the result:
    print_r($y);
    /* Result: Array ( [0] => Hello [1] => World! ) */
?>
```

Les fonctions essentielles de PHP: chaînes de caractères

- Pour concaténer, ou combiner, deux chaînes de caractères, vous pouvez utiliser l'opérateur « . »

```
<?php
    $x = "Hello";
    $y = "World";
    $z = $x . $y;
    echo $z;
?>
```

- Vous pouvez renvoyer une plage de caractères en utilisant la fonction **substr()**. Spécifiez l'indice de départ et le nombre de caractères que vous souhaitez renvoyer.

Exemple:

```
<?php
    $x = "Hello World!";
    echo substr($x, 6, 5);
?>
```

Les fonctions essentielles de PHP: les dates

La fonction PHP `date()` est utilisée pour formater une date et/ou une heure.

Voici quelques caractères couramment utilisés pour les dates :

- **d**: Représente le jour du mois (01 à 31)
- **m**: Représente un mois (01 à 12)
- **Y**: Représente une année (en quatre chiffres)

Voici quelques caractères couramment utilisés pour les heures :

- **H**: format 24 heures d'une heure (00 à 23)
- **h**: Format 12 heures d'une heure avec des zéros en tête (01 à 12)
- **i**: Minutes
- **s**: Secondes

D'autres caractères, comme `"/`, `"."` ou `"-"` peuvent également être insérés entre les caractères pour ajouter un formatage supplémentaire. L'exemple ci-dessous présente la date du jour de trois manières différentes :

```
<?php
echo "Today is " . date("Y/m/d") . "<br>";
echo "Today is " . date("Y.m.d") . "<br>";
echo "Today is " . date("Y-m-d") . "<br>";
echo "Today is " . date("Y-m-d H:i:s");

?>
```

Les fonctions essentielles de PHP: les dates

La fonction PHP **strtotime()** est utilisée pour convertir une chaîne de date lisible par l'homme en un timestamp Unix (le nombre de secondes depuis le 1er janvier 1970 00:00:00 GMT).

Exemples:

```
<?php
$d = strtotime("tomorrow");
echo date("Y-m-d h:i:sa", $d) . "<br>";

$d = strtotime("next Saturday");
echo date("Y-m-d h:i:sa", $d) . "<br>";

$d = strtotime("+3 Months");
echo date("Y-m-d h:i:sa", $d) . "<br>";
?>
```

Utilisation des cookies

- Les cookies sont un mécanisme d'enregistrement d'informations sur le client (navigateur), et de lecture de ces infos.
- Les cookies font partie des en-têtes HTTP, ce qui impose qu'ils doivent être appelés avant tout affichage de texte.
- Tous les cookies envoyés au serveur par le client seront automatiquement inclus dans un tableau auto-global `$_COOKIE`
- Un cookie est créé avec la fonction `setcookie()` avec la syntaxe suivante: `setcookie(name, value, expire, path, domain, secure, httponly)`, seul le paramètre `name` est obligatoire. Tous les autres paramètres sont facultatifs.

L'exemple suivant crée un cookie nommé "user" avec la valeur "ALAMI AHMED". Le cookie expirera après 30 jours (86400 * 30). Nous récupérons ensuite la valeur du cookie "user" (en utilisant la variable globale `$_COOKIE`). Nous utilisons également la fonction `isset()` pour savoir si le cookie est défini :

```
<?php
    setcookie("user", "ALAMI AHMED", time() + (86400 * 30)); // 86400 = 1 day
    if(isset($_COOKIE["user"])) { echo $_COOKIE["user"]; }
?>
```

- Pour modifier un cookie, il suffit de définir (à nouveau) le cookie à l'aide de la fonction `setcookie()`
- Pour supprimer un cookie, utilisez la fonction `setcookie()` avec une date d'expiration passée.

Exemple:

```
<?php
    setcookie("user", "", time() - 3600);
?>
```

Utilisation des sessions

- Le support des sessions est un moyen de préserver des données entre plusieurs accès. Contrairement à un cookie, les données ne sont pas stockées sur l'ordinateur de l'utilisateur.
- Le support des sessions vous permet de stocker des données entre les requêtes dans le tableau super-globale `$_SESSION`.
- Pour utiliser les sessions, il faut que `session.auto_start` soit activé dans `php.ini` ou la fonction `session_start()` soit appelée.
- Les variables de type resource ne peuvent être stockées dans une session.

```
<?php
    session_start();
    $_SESSION["user"] = "ALAMI AHMED";
    echo $_SESSION["user"];
?>
```

Pour modifier une variable de session, il suffit de l'écraser.

Pour supprimer toutes les variables globales de la session et détruire la session, utilisez `session_unset()` et `session_destroy()`.

```
<?php
    session_start();
    session_unset(); // remove all session variables
    session_destroy(); // destroy the session
?>
```

Connexion à une base de données MySQL avec PDO

Avec PHP, vous pouvez vous connecter à des bases de données et les manipuler. MySQL est le système de base de données le plus populaire utilisé avec PHP.

Avant de pouvoir accéder aux données de la base de données MySQL, nous devons être en mesure de nous connecter au serveur. Le code ci-dessous permet de se connecter à la base de données « test » sur le serveur MySQL situé dans l'adresse « localhost », et avec le nom d'utilisateur « root » sans mot de passe.

```
$host = 'localhost';
$dbname = 'test';
$username = 'root';
$password = '';

try {
    $connexion = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    echo "Vous êtes maintenant connecté à $dbname sur $host.";
} catch (PDOException $e) {
    die("Impossible de se connecter à la base de donnée $dbname :" . $e->getMessage());
}
```

La connexion sera fermée automatiquement à la fin du script. Pour fermer la connexion avant, affecter la valeur null à l'objet de connexion.

Insertion, modification et suppression des données avec MySQL

Pour insérer, modifier, ou supprimer des données il suffit de passer la requête SQL correspondante à la fonction exec

```
//identifiants mysql
$host = "localhost";
$username = "root";
$password = "";
$database = "utilisateurs";
$nom = $_POST["nom"];
$email = $_POST["email"];

// Ouvrir une nouvelle connexion au serveur MySQL
$conn= new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
// Exécuter la requête d'insertion
$conn->exec("INSERT INTO users (nom, email) VALUES('$nom', '$email')");
```

Requêtes préparées

Une meilleure méthode d'insertion est l'utilisation des requêtes préparées. Par rapport à l'exécution directe des requêtes SQL, les requêtes préparées présentent trois avantages principaux :

- Les requêtes préparées réduisent le temps d'analyse car la préparation de la requête n'est effectuée qu'une seule fois.
- Les paramètres liés réduisent la bande passante vers le serveur, car il suffit d'envoyer les paramètres à chaque fois, et non l'ensemble de la requête.
- Les instructions préparées sont très utiles contre les injections SQL, car les valeurs des paramètres, qui sont transmises ultérieurement à l'aide d'un protocole différent, n'ont pas besoin d'être correctement échappées.

```
//identifiants mysql
$host = "localhost";
$username = "root";
$password = "";
$database = "utilisateurs";
$nom = $_POST["nom"];
$email = $_POST["email"];

//Ouvrir une nouvelle connexion au serveur MySQL
$connexionPDO = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
//préparer la requête d'insertion SQL
$stmtement = $connexionPDO->prepare("INSERT INTO users (nom, email) VALUES(:nom, :email)");
//Associer les valeurs et exécuter la requête d'insertion
$stmtement->bindValue(':nom', $nom);
$stmtement->bindValue(':email', $email);
$stmtement->execute();
```

Requêtes préparées: Exemple de modification et de suppression

```
//identifiants mysql
$host = "localhost";
$username = "root";
$password = "";
$database = "utilisateurs";
$nom = $_POST["nom"];
$id= $_POST["id"];

//Ouvrir une nouvelle connexion au serveur MySQL
$connexionPDO = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
//préparer la requête d'insertion SQL
$stmtement = $connexionPDO->prepare("UPDATE users SET nom = :nom WHERE id = :id");
//Associer les valeurs et exécuter la requête d'insertion
$stmtement->bindValue(':nom', $nom);
$stmtement->bindValue(':id', $id);
$stmtement->execute();

$stmtement = $connexionPDO->prepare("DELETE FROM users WHERE id = :id");
$stmtement->bindValue(':id', $id);
$stmtement->execute();
```

Récupération des données depuis MySQL

```
<?php
    $connexionPDO = new PDO("mysql:host=localhost;dbname=utilisateurs", "root", "");
    $statement = $connexionPDO->prepare("SELECT * FROM users");
    $statement->execute();
    $users = $statement->fetchAll(PDO::FETCH_ASSOC);
?>
<html>
    <head></head>
    <body>
        <table>
            <tr>
                <th>Nom</th>
                <th>Email</th>
            </tr>
            <?php foreach($users as $user): ?>
            <tr>
                <td><?php echo $user['nom']; ?></td>
                <td><?php echo $user['email']; ?></td>
            </tr>
            <?php endforeach; ?>
        </table>
    </body>
</html>
```